

第 15 章 Linux PCI 的中断处理

Linux PCI 的中断处理包含两部分内容，一部分是 PCI 设备使用 INTx 信号，包括 PCIe 设备使用 INTx 消息，向处理器提交的中断请求，这种中断请求方式也被称为 PCI 设备的传统中断请求；而另一部分是处理 MSI/MSI-X 中断机制。

Linux PCI 在处理传统中断请求时，需要考虑 PCI 总线的中断路由。本章将首先介绍 PCI 总线的中断路由，并在第 15.2 节介绍 MSI 和 MSI-X 中断机制，而不再详细介绍 PCI 设备的传统中断请求。

15.1 PCI 总线的中断路由

在多数 x86 处理器系统中，PCI 设备的 INTA~D# 四个中断请求信号与 LPC 接口提供的外部引脚 PIRQA~D# 相连，之后 PIRQA~D# 与 I/O APIC 的中断请求信号 IRQ_PIN16~19# 相连。如果 PCIe 设备没有使用 MSI 中断请求机制，而是使用了 Legacy INTx 方式^①模拟 INTA~D# 信号时，这些 Assert INTx 和 Deassert INTx 消息也由 Chipset 处理，并由 Chipset 将这些消息转换为一根硬件引脚，然后将这个硬件引脚与 I/O APIC 的中断输入引脚相连。其连接关系如图 15-1 所示。I/O APIC 最终使用 REDIR_TBL 表，将来自输入引脚的中断请求发送至 Local APIC，并由 CPU 进一步处理这个中断请求。

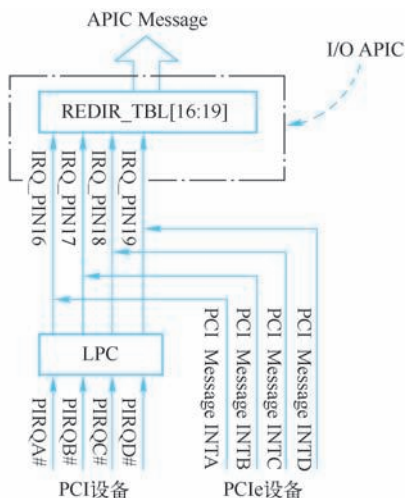


图 15-1 I/O APIC 如何处理 PCI 设备的中断请求

① I/O APIC 将这种方式称为 PCI Message。PCIe 设备可以通过 INTx 中断消息报文，向 I/O APIC 提交中断请求，详见第 6.3.4 节。

本书并不关心 I/O APIC 如何使用 APIC Message 将中断消息传递给 Local APIC，而重点关注 PCI 和 PCIe 设备使用的中断信号与 I/O APIC 输入引脚 IRQ_PIN16~19 的连接关系。如图 15-1 所示，LPC 的 PIRQA~D# 分别与 IRQ_PIN16~19 对应，但是 PCI 设备的 INTA~D# 与 PIRQA~D# 的连接关系并不是唯一的，图 15-1 所示的 PCI 设备与中断控制器连接方法只是其中一种连接方法。

而无论硬件采用何种连接结构，系统软件都需要能够正确识别是哪个 PCI 设备发出的中断请求，为此系统软件使用 PCI 中断路由表（PCI Interrupt Routing Table）记录 PCI 设备使用的 INTA~D# 与 I/O APIC 中断输入引脚 IRQ16~19 的对应关系。

如果在 x86 处理器系统中存在 Switch，而这个 Switch 的每一个端口都相当于一个虚拟 PCI 桥，当 Switch 的下游端口收到 PCI Message INTx 消息后，将通过虚拟 PCI 桥向上传递这个消息。值得注意的是，虚拟 PCI 桥可能会改变 PCI Message INTx 消息，如将设备号为 1 的 INTA# 消息转换为 INTB# 消息。在虚拟 PCI 桥中，Primary 总线和 Secondary 总线 PCI Message INTx 消息的映射关系如表 15-1 所示。

表 15-1 虚拟 PCI 桥 Primary 总线与 Secondary 总线间 INTx 消息间的映射关系

设备号	PCI 桥 Secondary 总线的虚拟中断信号 INTx#	PCI 桥 Primary 总线的虚拟中断信号 INTx#
0, 4, 8, 12, 16, 20, 24, 28	INTA#	INTA#
	INTB#	INTB#
	INTC#	INTC#
	INTD#	INTD#
1, 5, 9, 13, 17, 21, 25, 29	INTA#	INTB#
	INTB#	INTC#
	INTC#	INTD#
	INTD#	INTA#
2, 6, 10, 14, 18, 22, 26, 30	INTA#	INTC#
	INTB#	INTD#
	INTC#	INTA#
	INTD#	INTB#
3, 7, 11, 15, 19, 23, 27, 31	INTA#	INTD#
	INTB#	INTA#
	INTC#	INTB#
	INTD#	INTC#

PCIe 设备发送的 PCI Message INTx 消息首先到达虚拟 PCI 桥的 Secondary 总线，之后虚拟 PCI 桥根据 PCIe 设备的设备号将这些 PCI Message INTx 消息转换为 Primary 总线合适的虚拟中断信号。如设备号为 1 的 PCIe 设备使用 PCI Message INTA 消息进行中断请求时，该消息在通过虚拟 PCI 桥后，将被转换为 PCI Message INTB 消息，然后继续传递该消息报文，最终 PCI Message INTx 消息将到达 RC，并由 RC 将该消息报文转换为虚拟中断信号 INTx，并与 I/O APIC 的中断请求引脚 IRQ_PIN16~19 相连。

然而直接使用 PCIe 总线提供的标准方法会带来一些问题。因为一条 PCIe 链路只能挂接

一个 EP，这个 EP 的设备号通常为 0，而这些设备使用的虚拟中断信号多为 INTA#，因此这些 PCIe 设备通过 Switch 的虚拟 PCI-to-PCI 桥进行中断路由后，将使用虚拟中断信号 INTA#，并与 I/O APIC 的 IRQ_PIN16 引脚相连，并不会使用其他 IRQ_PIN 引脚，这造成了 IRQ_PIN16 的负载过重。其连接拓扑结构如图 15-2 所示。

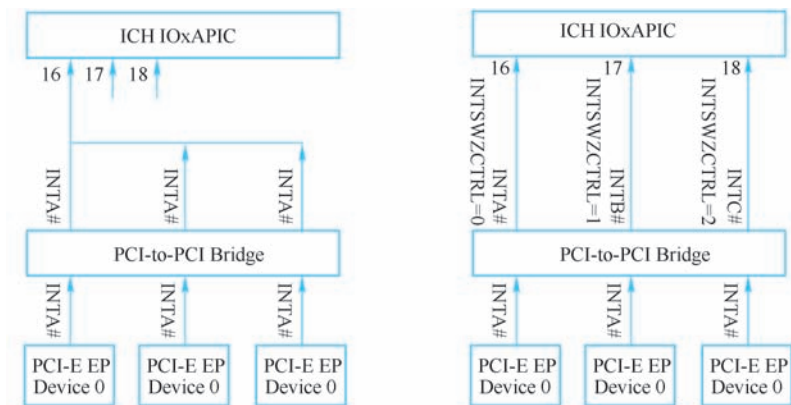


图 15-2 PCI Message 中断路由

如上图所示，PCIe 设备使用的 INTx 中断请求都最终使用 I/O APIC 的 IRQ_PIN16 引脚，从而造成了这个引脚所申请的中断过于密集，因此采用这种中断路由方法并不合理。为此 Intel 在 5000 系列的 Chipset 中使用了 Interrupt Swizzling 技术将这些来自 PCIe 设备的中断请求平均分配到 I/O APIC 的 IRQ_PIN16~19 引脚中。

在图 15-2 中，Chipset 设置了一个 INTSWZCTRL 寄存器，通过这些寄存器可以将 PCIe 设备提交的中断请求均衡地发送至 I/O APIC 中。如果一个 EP 对应的 INTSWZCTRL 位为 0，则该设备的 INTA# 将与 IRQ_PIN16 相连；如果为 1，将与 IRQ_PIN17 相连，并以此类推，最终实现中断请求的负载均衡。

在一个 x86 处理器系统中，PCI 设备或者 PCIe 设备使用的中断信号 INTA~D# 与 I/O APIC 的 IRQ_PIN16~19 之间的对应关系并不明确，各个厂商完全可以按照需要定制其映射关系。这为系统软件的设计带来了不小的麻烦。为此 BIOS 为系统软件提供了一个 PCI 中断路由表，存放这个映射关系，ACPI 规范将这个中断路由表存放在 DSDT 中。

值得注意的是，每一个 HOST 主桥和每一条 PCI 总线都含有一个中断路由表。在讲述 PCI 中断路由表之前，我们简要回顾 Linux 系统如何为 PCI 设备分配中断向量。

15.1.1 PCI 设备如何获取 irq 号

在 Linux 系统中，PCI 设备使用的 irq 号存放在 `pdev→irq` 参数中，该参数在 Linux 设备驱动程序进行初始化时，由 `pci_enable_device` 函数设置。本书在第 12.3.2 节曾简要介绍过这个函数，下文进一步说明如何使用该函数设置 PCI 设备的 irq 号。`pci_enable_device` 函数将依次调用 `__pci_enable_device_flags→do_pci_enable_device→pcibios_enable_device` 函数设置 PCI 设备使用的 irq 号。

`pcibios_enable_device` 函数将调用 `pcibios_enable_irq` 函数，设置 PCI 设备使用的 irq 号。如果处理器系统使能了 ACPI 机制，`pcibios_enable_irq` 函数将被赋值为 `acpi_pci_irq_enable`。`acpi_pci_irq_enable` 函数在 `./drivers/acpi/pci_irq.c` 文件中，其实现过程如源代码 15-1 所示。

源代码 15-1 acpi_pci_irq_enable 函数

```

int acpi_pci_irq_enable(struct pci_dev *dev)
{
    struct acpi_prt_entry *entry;
    int gsi;
    u8 pin;
    int triggering = ACPI_LEVEL_SENSITIVE;
    int polarity = ACPI_ACTIVE_LOW;
    char *link = NULL;
    char link_desc[16];
    int rc;

    pin = dev->pin;
    if (! pin) {
        ACPI_DEBUG_PRINT((ACPI_DB_INFO,
            "No interrupt pin configured for device %s\n",
            pci_name(dev)));
        return 0;
    }

    entry = acpi_pci_irq_lookup(dev, pin);
    if (! entry) {
        /*
         * IDE legacy mode controller IRQs are magic. Why do compat
         * extensions always make such a nasty mess.
         */
        if (dev->class >> 8 == PCI_CLASS_STORAGE_IDE &&
            (dev->class & 0x05) == 0)
            return 0;
    }

    if (entry) {
        if (entry->link)
            gsi = acpi_pci_link_allocate_irq(entry->link,
                entry->index,
                &triggering, &polarity,
                &link);
        else
            gsi = entry->index;
    } else
        gsi = -1;

    ...
}

```

```

if (gsi < 0) {
    dev_warn(&dev->dev, "PCI INT %c: no GSI", pin_name(pin));
    /* Interrupt Line values above 0xF are forbidden */
    if (dev->irq > 0 && (dev->irq <= 0xF)) {
        printk(" - using IRQ %d\n", dev->irq);
        acpi_register_gsi(&dev->dev, dev->irq,
            ACPI_LEVEL_SENSITIVE,
            ACPI_ACTIVE_LOW);
        return 0;
    } else {
        printk("\n");
        return 0;
    }
}

rc = acpi_register_gsi(&dev->dev, gsi, triggering, polarity);
if (rc < 0) {
    dev_warn(&dev->dev, "PCI INT %c: failed to register GSI\n",
        pin_name(pin));
    return rc;
}
dev->irq = rc;
...
return 0;
}

```

该函数首先调用 `acpi_pci_irq_lookup→acpi_pci_irq_find_prt_entry` 函数，从 `acpi_prt_list` 链表中获得一个 `acpi_prt_entry` 结构的 Entry。在 `acpi_prt_list` 链表中存放 PCI 总线的中断路由表，本章将在第 15.1.2 节进一步介绍该表。在这个 Entry 中，存放 PCI 设备使用的 Segment、Bus、Device 和 Function 号，PCI 设备使用的中断请求信号（INTA#~INTD#）和 GSI（Global System Interrupt）号。

这段程序在获得 Entry 后，将判断 `Entry→link` 是否为空，如果为空，表示当前 x86 处理器系统使用 I/O APIC 管理外部中断，而不是使用 8259A。在 Intel 的 ICH9 中集成了两个中断控制器，一个是 8259A，另一个是 I/O APIC。Linux x86 通过软件配置，决定究竟使用哪个中断控制器，在绝大多数情况下，Linux x86 使用 I/O APIC 而不是 8259A 管理外部中断请求 200。本章不再关心 8259A 中断控制器，因此也不再关心 `Entry→link` 不为空的处理情况^①。

这段程序在获得 GSI 号之后，将调用 `acpi_register_gsi` 函数，将 GSI 号转换为系统软件使用的 irq 号。`acpi_register_gsi` 函数使用三个入口参数，分别为 GSI 号、中断触发方式和采用

① Linux 1A 在引导时可以加入“noapic”参数关闭 I/O APIC，此时处理器系统将使用 8259A 中断控制器。

电平触发时的极性。其中 PCI 设备使用低电平触发方式。

acpi_register_gsi 函数执行完毕后，将为 PCI 设备分配一个 irq 号，这个 irq 号是系统软件使用的，之后 PCI 设备的驱动程序可以使用 request_irq 函数将中断服务例程与 irq 号建立映射关系；该函数还将设置 I/O APIC 的 REDIR_TBL 表，将 GSI 号与 REDIR_TBL 表中的中断向量建立对应关系，同时初始化与操作系统相关的 irq 结构^①。为了深入理解 acpi_register_gsi 函数，读者需要理解 GSI 号、I/O APIC 的 REDIR_TBL 表、IRQ_PIN 引脚和 Linux 使用的 irq 号之间的对应关系。

GSI 号是 ACPI 规范引入的，用于记录 I/O APIC 的 IRQ_PIN 引脚号的参数。如果 x86 处理器系统使用 I/O APIC 管理外部中断请求，而且在这个处理器系统中具有多个 I/O APIC 控制器，那么 GSI 号和 I/O APIC 中断引脚号的对应关系如图 15-3 所示。

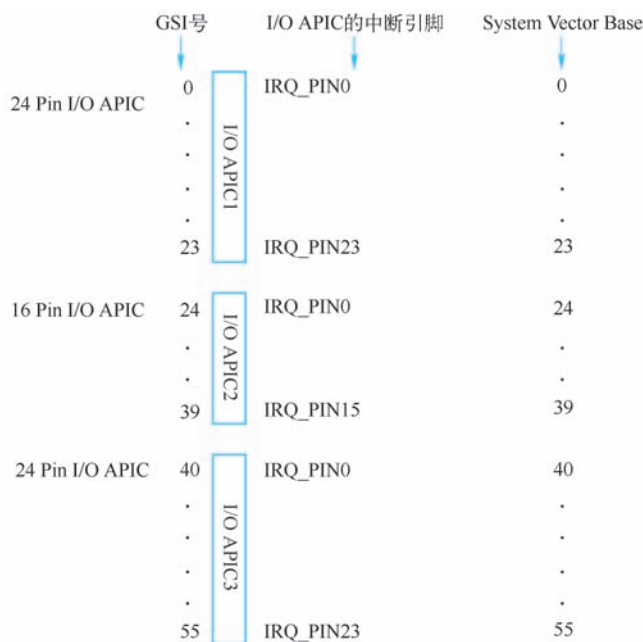


图 15-3 GSI 号和 I/O APIC 中断引脚号的对应关系

假设在一个 x86 处理器系统中存在 3 个 I/O APIC，其中有两个 I/O APIC 的外部中断引脚数为 24 根，另外一个 I/O APIC 的外部中断引脚数为 16 根。其中 GSI 号的 0~23 与 I/O APIC1 的 IRQ_PIN0~23 对应；GSI 号的 24~39 与 I/O APIC2 的 IRQ_PIN0~15 对应；而 GSI 号的 40~55 与 I/O APIC3 的 IRQ_PIN0~23 对应。ACPI 规范为统一起见使用 GSI 号描述外部设备与 I/O APIC 中断引脚的连接关系。

I/O APIC 的 IRQ_PIN 引脚与外部设备的中断请求引脚相连，如 I/O APIC1 的 IRQ_PIN16 与某个 PCI 设备的 INTA# 相连。值得注意的是，PCI 设备的 INTA# 信号首先与 LPC 的 PIRQA# 信号相连，而 PIRQA# 信号再与 I/O APIC1 的 IRQ_PIN16 相连。其中 I/O APIC 集成在 ICH 中，因此这些 IRQ_PIN 引脚并没有从 ICH 中引出。

① 在 Linux 系统中，该结构为 irq_desc，该结构记录与 irq 号相关的信号。

REDIR_TBL 表中存放对 IRQ_PIN 引脚的描述，一个 I/O APIC 具有多少个 IRQ_PIN 引脚，REDIR_TBL 表就由多少项组成。该表的每一个 Entry 由多个字段组成，本节仅对这个 Entry 的 Vector 字段感兴趣，Vector 字段是这个 Entry 的第 7~0 位，存放对应 IRQ_PIN 引脚使用的中断向量。

在 Linux 系统中，与 IRQ_PIN 引脚对应的中断向量由 acpi_register_gsi 函数设置，当 x86 处理器系统使用 I/O APIC 管理外部中断时，acpi_register_gsi 函数将调用 mp_register_gsi 函数。mp_register_gsi 函数在 ./drivers/acpi/boot.c 文件中定义，其实现机制如源代码 15-2 所示。我们假定在 Linux 系统中使能了 CONFIG_X86_32 选项。

源代码 15-2 mp_register_gsi 函数

```
int mp_register_gsi(struct device *dev, u32 gsi, int trigger, int polarity)
{
    int ioapic;
    int ioapic_pin;
    ...
    ioapic = mp_find_ioapic(gsi);
    if (ioapic < 0) {
        printk(KERN_WARNING "No IOAPIC for GSI %u\n", gsi);
        return gsi;
    }

    ioapic_pin = mp_find_ioapic_pin(ioapic, gsi);

#ifdef CONFIG_X86_32
    if (ioapic_renumber_irq)
        gsi = ioapic_renumber_irq(ioapic, gsi);
#endif

    if (ioapic_pin > MP_MAX_IOAPIC_PIN) {
        printk(KERN_ERR "Invalid reference to IOAPIC pin "
            "%d-%d\n", mp_ioapics[ioapic].apicid,
            ioapic_pin);
        return gsi;
    }

    if (enable_update_mptable)
        mp_config_acpi_gsi(dev, gsi, trigger, polarity);

    set_io_apic_irq_attr(&irq_attr, ioapic, ioapic_pin,
```

```

        trigger == ACPI_EDGE_SENSITIVE ? 0 : 1,
        polarity == ACPI_ACTIVE_HIGH ? 0 : 1);
io_apic_set_pci_routing(dev, gsi, &irq_attr);

return gsi;
}

```

这段程序首先根据 GSI 号，使用 `mp_find_ioapic` 和 `mp_find_ioapic_pin` 函数，确定当前 PCI 设备与 I/O APIC 中断控制器的哪个 IRQ_PIN 引脚相连（GSI 号与 I/O APIC 和 IRQ_PIN 引脚的对应关系如图 15-1 所示）。

然后 `mp_register_gsi` 函数调用 `io_apic_set_pci_routing` 函数设置 I/O APIC 中的寄存器。在 Linux x86 的源代码中，`mp_register_gsi` 函数调用 `io_apic_set_pci_routing` 函数时，有一个并不恰当的处理，在 `mp_register_gsi` 函数中使用 GSI 号作为 `io_apic_set_pci_routing` 函数的第二个入口参数，但是 `io_apic_set_pci_routing` 函数要求的这个输入参数是 irq 号。

在 Linux x86 系统中，irq 号是一个纯软件^①概念，而这段代码的作用实际上是令 GSI 号直接等于 irq 号。笔者认为这种方法并不十分恰当，因为 GSI 号用来描述 I/O APIC 的 IRQ_PIN 输入引脚，而 irq 号是设备驱动程序用来挂接中断服务例程的。

本节在此强调这个问题，主要为了读者辨明 GSI 号和 irq 号的关系，目前在 Linux x86 系统中，PCI 设备使用的 GSI 号与 irq 号采用了“直接相等”^②的一一映射关系，实际上，GSI 号并不等同于 irq 号。在系统软件的实现中，两者只要建立一一映射的对应关系即可，并不一定要“直接相等”。还有一点需要提醒读者注意，就是不同的 PCI 设备可以共享同一个 GSI 号，即共享 I/O APIC 的一个 IRQ_PIN 引脚，从而在 Linux 系统中共享同一个 irq 号。

`io_apic_set_pci_routing` 函数调用 `__io_apic_set_pci_routing→setup_IO_APIC_irq` 操作 I/O APIC 中的寄存器。`setup_IO_APIC_irq` 是一个重要函数，如源代码 15-3 所示。

源代码 15-3 `setup_IO_APIC_irq` 函数

```

static void setup_IO_APIC_irq(int apic_id, int pin, unsigned int irq,
                             struct irq_desc *desc, int trigger, int polarity)
{
    struct irq_cfg *cfg;
    struct IO_APIC_route_entry entry;
    unsigned int dest;
    ...
    cfg = desc->chip_data;
    if (assign_irq_vector(irq, cfg, TARGET_CPUS))
        return;
    ...
    if (setup_ioapic_entry(mp_ioapics[apic_id].apicid, irq, &entry,

```

① 如果 Linux x86 并没有使用 8825A 作为中断控制器，irp 号和中断向量并没有直接的对应关系。

② 如果在一个处理器系统中，irp 号大于 16，那么 irp 号等于 GSI 号。


```

        dest, trigger, polarity, cfg->vector, pin)) {
    printk("Failed to setup ioapic entry for ioapic %d, pin %d\n",
        mp_ioapics[apic_id].apicid, pin);
    __clear_irq_vector(irq, cfg);
    return;
}

ioapic_register_intr(irq, desc, trigger);
if (irq < NR_IRQS_LEGACY)
    disable_8259A_irq(irq);

ioapic_write_entry(apic_id, pin, entry);
}

```

该函数首先调用 `assign_irq_vector→_assign_irq_vector` 函数将外部设备使用的 GSI 号与 I/O APIC 中 REDIR_TBL 表建立联系，并将其结果记录到 CPU 的 `vector_irq` 表中。这个步骤非常重要，在 Linux x86 系统中，如果存在多个 CPU，那么每一个 CPU 都有一个 `vector_irq` 表，这张表中包含了 `vector` 号与 `irq` 号的对应关系。这张表也是处理器硬件与系统软件联系的桥梁。

处理器硬件并不知道 `irq` 号的存在，而仅仅知道 `vector` 号，而 Linux x86 系统使用的是 `irq` 号。在处理外部中断请求时，Linux 系统需要通过 `vector_irq` 表将 `vector` 号转换为 `irq` 号才能通过 `irq_desc` 表找到相关设备的中断服务例程。

`setup_ioapic_entry` 函数将初始化 `entry` 参数。该参数是一个 `IO_APIC_route_entry` 类型的结构。而 `ioapic_register_intr` 函数调用 `set_irq_chip_and_handler_name` 函数设置 `irq_desc[irq]` 变量，并将这个变量的 `chip` 参数设置为 `ioapic_chip`，`handle_irq` 参数设置为 `handle_fasteoi_irq`，这个步骤对于 Linux x86 中断处理系统非常重要。

`ioapic_write_entry` 函数将保存在 `entry` 参数中的数据写入到与 GSI 号对应的 REDIR_TBL 表中，该函数将直接操作 I/O APIC 的寄存器。

由以上描述，我们可以发现当 `acpi_pci_irq_enable` 函数执行完毕后，Linux 系统将 GSI 号与 `irq` 号建立映射关系，同时又将 `irq` 号与 I/O APIC 中的 `vector` 号进行映射，并将这个映射关系记录到 `vector_irq` 表中，这个映射表由操作系统使用。之后该程序还将初始化 I/O APIC 的 REDIR_TBL 表，将 PCI 设备使用的 GSI 号与 I/O APIC 的 `vector` 号联系在一起。

在 x86 处理器系统中，PCI 设备的 INTx 引脚首先与 LPC 的 PIRQA~H 引脚直接相连，而 LPC 中的 PIRQA~H 引脚将与 I/O APIC 的 IRQ_PIN16~23 引脚相连。当 PCI 设备通过 INTx 引脚提交中断请求时，最终将传递到 IRQ_PIN16~23 引脚。而 I/O APIC 接收到这个中断请求后，将根据 REDIR_TBL 表与“IRQ_PIN16~23 引脚”对应的 Entry 向 Local APIC 发送中断请求消息，处理器通过 Local APIC 收到这个中断请求后，将执行中断处理程序进一步处理这个来自 PCI 设备的中断请求。

Linux x86 系统使用 `do_IRQ` 函数处理外部中断请求，该函数在 `./arch/x86/kernel/irq.c` 文件中，如源代码 15-4 所示。

源代码 15-4 do_IRQ 函数

```

unsigned int __irq_entry do_IRQ(struct pt_regs *regs)
{
    struct pt_regs *old_regs = set_irq_regs(regs);

    /* high bit used in ret_from_code */
    unsigned vector = ~regs->orig_ax;
    unsigned irq;

    exit_idle();
    irq_enter();

    irq = __get_cpu_var(vector_irq)[vector];

    if (!handle_irq(irq, regs)) {
        ack_APIC_irq();

        if (printk_ratelimit())
            pr_emerg("%s: %d. %d No irq handler for vector (irq %d)\n",
                    __func__, smp_processor_id(), vector, irq);
    }

    irq_exit();

    set_irq_regs(old_regs);
    return 1;
}

```

do_IRQ 函数首先获得 vector 号，这个 vector 号由 I/O APIC 传递给 Local APIC，并与某个 IRQ_PIN 引脚对应，其描述在 I/O APIC 的 REDIR_TBL 表中。vector 号是一个硬件概念，x86 处理器系统在处理外部中断请求时，仅仅知道 vector 号的存在，而不知道 irq 号。

Linux x86 系统通过 vector_irq 表，将 vector 号转换为 irq 号，之后执行 handle_irq 函数进一步处理这个中断请求。对于 PCI 设备，这个 handle_irq 函数将调用 handle_fasteoi_irq 函数，而 handle_fasteoi_irq 函数将最终执行 PCI 设备使用的中断服务例程。handle_fasteoi_irq 函数的源代码在 ./kernel/irq/chip.c 文件中，本节对该函数不做进一步分析。

在 PCI 设备的 Linux 驱动程序中，将使用 request_irq 函数将其中断服务例程挂接到系统中断服务处理程序中。

15.1.2 PCI 中断路由表

上节简要介绍了 PCI 设备如何获取中断向量。由上文所述，PCI 设备在获取中断向量之前需要从 acpi_prt_list 链表获得 GSI 号，在 acpi_prt_list 链表中存放 PCI 总线的中断路由表，

而这个中断路由表中存放 PCI 设备所使用的 GSI 号。

这个 PCI 中断路由表由 BIOS 提供，如果 x86 处理器系统支持 ACPI 机制，这个中断路由表存在于 DSDT. dsl 文件中，如源代码 15-5 所示。ACPI 规范使用 ASL 语言描述 PCI 中断路由表。

源代码 15-5 DSDT 表中的 PCI 中断路由表

```
Device (PCI0)
{
    ...
    Method (_PRT, 0, NotSerialized)
    {
        If (LEqual (GPIC, Zero))
        {
            Package (0x04) { 0x0001FFFF, 0x00, \_SB.PCI0.LPC.LNKA, 0x00 },
            Package (0x04) { 0x0001FFFF, 0x01, \_SB.PCI0.LPC.LNKB, 0x00 },
            Package (0x04) { 0x0001FFFF, 0x02, \_SB.PCI0.LPC.LNKC, 0x00 },
            Package (0x04) { 0x0001FFFF, 0x03, \_SB.PCI0.LPC.LNKD, 0x00 },
            ...
        }
        Else
        {
            Return (Package (0x47) {
                ...
                Package (0x04) { 0x001CFFFF, Zero, Zero, 0x11 },
                Package (0x04) { 0x001CFFFF, One, Zero, 0x10 },
                Package (0x04) { 0x001CFFFF, 0x02, Zero, 0x12 },
                Package (0x04) { 0x001CFFFF, 0x03, Zero, 0x13 },

                Package (0x04) { 0x001DFFFF, Zero, Zero, 0x17 },
                Package (0x04) { 0x001DFFFF, One, Zero, 0x13 },
                Package (0x04) { 0x001DFFFF, 0x02, Zero, 0x12 },
                Package (0x04) { 0x001DFFFF, 0x03, Zero, 0x10 },
                ... })
        }
    }
}
```

在以上源代码中，_PRT 存放 x86 处理器系统 PCI 总线 0 的中断路由表，在 x86 处理器体系结构中，每一条 PCI 总线都有一个中断路由表，因此在 DSDT 中，将存在多个中断路由表。在以上源代码中，我们仅列出 PCI 总线 0 使用的中断路由表，即 RC 使用的中断路由表，在一个处理器系统中还可能还有其他中断路由表，如 PCIe 桥使用的中断路由表等。

在以上源代码中，首先判断 GPIC 是否为 0，如果为 0 表示当前 x86 处理器系统使用 PIC

模式，即使用 8259A 中断控制器管理外部中断，在第 15.1.3 节将介绍这种情况；如果为 1 表示当前 x86 处理器系统使用 I/O APIC 管理外部中断，此时“Package (0x04)”[⊖]中含有四个参数，这四个参数的定义如表 15-2 所示。

表 15-2 PCI 中断路由表使用的参数

参数	类型	描 述
Address	DWORD	设备地址，其中高两个字节表示 PCI 设备的 Device 号，低两个字节表示 PCI 设备的 Function 号，如果低两字节为 0xFFFF 表示全部 Function 号
Pin	Byte	其中 0~3 分别与 INTA~D#引脚#对应
Source	Name Path 或者 Byte	该字段为 0 表示使用 GSI 号描述 PCI 设备使用的中断资源，否则该字段存放该设备与 LPC 的 PIRQ 引脚的相连关系，如 LPC 的 PIRQA 信号连接，在第 15.1.3 节将讲述 LPC 的 PIRQ 引脚的描述
Source Index	DWORD	Source 字段为 0 时，该字段存放 PCI 设备使用的 GSI 号

通过以上描述，发现“Package (0x04) { 0x0001FFFF, 0x00, _SB.PCI0.LPC.LNKA, 0x00 }”[⊖]的含义为，PCI 总线 0 的某个设备，其 Device 号为 0x01，而且这个设备的 INTA#引脚与 LPC 的 PIRQA 相连，INTB#引脚与 PIRQB 相连，INTC#引脚与 PIRQC 相连，而 INTD#引脚与 PIRQD 相连。

而“Package (0x04) { 0x001CFFFF, Zero, Zero, 0x11 }...”[⊖]这段代码的含义为，PCI 总线 0 的某个 PCI 设备，其 Device 号为 0x1C，而且这个设备的 INTA#引脚使用的 GSI 号为 0x11；这个 PCI 设备的 INTB#引脚使用的 GSI 号为 0x10；这个 PCI 设备的 INTC#使用的 GSI 号为 0x12，这个 PCI 设备的 INTD#引脚使用的 GSI 号为 0x13。

Linux x86 系统进行初始化时，将_PRT 表加载到 acpi_prt_list 链表中，操作系统首先执行 acpi_pci_root_init 函数，之后调用 acpi_device_probe→acpi_bus_driver_init→acpi_pci_root_add 函数。acpi_pci_root_add 函数将调用 acpi_pci_irq_add_prt→acpi_pci_irq_add_entry 函数将_PRT 表中的中断路由表的每一个 Entry 加载到 acpi_prt_list 链表。

通过上文的分析，可以发现在每一个 PCI 桥中，包括 Switch 的虚拟 PCI 桥中都有一个中断路由表，因此 acpi_pci_root_add 还会调用 acpi_pci_bridge_scan 函数分析并加载每一个 PCI 桥的中断路由表。对 Linux x86 系统初始化 PCI 中断路由表感兴趣的读者可以自行分析这段代码，本节对此不做进一步介绍。

在 Linux x86 系统中，PCI 设备在获取 irq 号时，将从这个链表中获得 GSI 号，从而最终获得 irq 号，具体过程见第 15.1.1 节。

15.1.3 PCI 插槽使用的 irq 号

在 x86 处理器系统中，还有一类特殊的 PCI 设备，即 PCI 插槽。PCI 插槽无法确定其上的 PCI 设备如何使用 INTA#~INTD#信号，因此必须处理全部中断请求引脚，而在其上的

⊖ 对应 Else 之后的这段代码。

⊖ 使用 8859A 中断控制器的情况。

⊖ 使用 APIC 中断控制器的情况。

PCI 设备有选择地使用这些信号。

PCI 插槽使用的中断请求信号将与 LPC 的 PIRQA~F 相连，如果处理器系统使能了 I/O APIC，LPC 的这些中断请求引脚将与 IRQ_PIN16~23 相连，否则中断控制器 8259A 将管理这些中断引脚。在 ACPI 表中含有对这些 PCI 插槽中断请求信号的描述，这些描述主要针对处理器系统没有使用 I/O APIC 的处理情况，如源代码 15-6 所示。

源代码 15-6 PCI 插槽使用中断请求信号

```

Device (LPC)
{
...
    Device (LNKA)
    {
        Name (_HID, EisaId ("PNP0C0F"))
        Name (_UID, 0x01)
        Method (_STA, 0, NotSerialized)
        {
            If (And (PIRA, 0x80))
            {
                Return (0x09)
            }
            Else
            {
                Return (0x0B)
            }
        }
        Method (_DIS, 0, NotSerialized)
        {
            Or (PIRA, 0x80, PIRA)
        }
        Method (_CRS, 0, NotSerialized)
        {
            Name (BUF0, ResourceTemplate ()
            {
                IRQ (Level, ActiveLow, Shared, _Y02)
                {0}
            })
            CreateWordField (BUF0, \_SB.PCI0.LPC.LNKA._CRS._Y02._INT, IRQW)
            If (And (PIRA, 0x80))
            {
                Store (Zero, Local0)
            }
            Else

```

```

        {
            Store ( One, Local0)
        }
        ShiftLeft ( Local0, And ( PIRA, 0x0F), IRQW)
        Return ( BUF0)
    }
    Name ( _PRS, ResourceTemplate ( )
    {
        IRQ ( Level, ActiveLow, Shared, )
        { 3,4,5,7,9,10,11,12}
    } )
    Method ( _SRS, 1, NotSerialized)
    {
        CreateWordField ( Arg0, 0x01, IRQW)
        FindSetRightBit ( IRQW, Local0)
        If ( LNotEqual ( IRQW, Zero))
        {
            And ( Local0, 0x7F, Local0)
            Decrement ( Local0)
        }
        Else
        {
            Or ( Local0, 0x80, Local0)
        }
        Store ( Local0, PIRA)
    }
}

```

在 ACPI 规范中，PCI 插槽的中断请求信号的标识符“PNP0C0F”。在这段源代码中 LNKA 与 LPC 的 PIRQA 引脚对应，这段代码的作用是描述 LPC 的 PIRQA 引脚。在 ICH 中，使用 PIRQA_ROUT 寄存器描述 PIRQA 引脚。在以上这段源程序中，“_STA”、“_DIS”、“_CRS”和“_PRS”可以操作 PIRQA_ROUT 寄存器，具体含义如下所示。

_STA 用来测试当前 PIRQA 引脚的状态，这段代码判断 PIRQA_ROUT 寄存器的第 7 位是否为 1，如果为 1 表示当前 PIRQ 引脚并没有与 8259A 相连，此时 I/O APIC 将管理该引脚，_STA 将返回 0x09 表示 PIRQA 没有与 8259A 相连；否则返回 0x0B，表示 PIRQA 与 8259A 相连。_STA 的返回值在 ACPI 规范中具有明确的定义。

_DIS 用来关闭 PIRQA 引脚与 8259A 的联系，即使用 I/O APIC 管理该引脚。_DIS 的作用是将 PIRQA_ROUT 寄存器的第 7 位置 1。

_CRS 用来获得当前资源的描述，对于 PIRQA 引脚而言，这段描述表示 PIRQA 引脚使用“低电平有效的共享中断请求”，随后通过 PIRQ[A]_ROUT 寄存器的最高位判断，该中断信号是由 8259A 中断控制器还是 APIC 中断控制器接管，最后将 IRQW 根据 PIRQ[A]_

ROUT 寄存器的 IRQ Routing 字段赋值, IRQ Routing 字段可以使用的资源在 {3, 4, 5, 7, 9, 10, 11, 12} 集合中。

_PRS 描述 PCI 插槽的中断请求信号可能使用的中断资源, 对于 PIRQA 而言, 可能使用的 irq 号为 {3, 4, 5, 7, 9, 10, 11, 12}。这些 irq 号由 x86 处理器系统规定, 这些 irq 号与 ISA 总线兼容, 如果一个系统使用了 I/O APIC, 这些规定将不再有效。

在 Linux 系统中, acpi_pci_link_init 函数处理 PCI 插槽的中断请求, 该函数在 ./drivers/acpi/pci_link.c 文件中, 其实现如源代码 15-7 所示。

源代码 15-7 acpi_pci_link_init 函数

```
static int __init acpi_pci_link_init( void)
{
    ...

    if ( acpi_bus_register_driver(&acpi_pci_link_driver) < 0)
        return -ENODEV;

    return 0;
}

subsys_initcall( acpi_pci_link_init );
```

acpi_pci_link_init 函数调用 acpi_bus_register_driver→...→acpi_pci_link_add 函数将 LPC 的 PIRQA~H 引脚与 irq 号对应在一起。acpi_pci_link_add 函数的执行过程较为简单, 首先该函数调用 acpi_pci_link_get_possible 函数, 运行 _PRS 代码获得 {3, 4, 5, 7, 9, 10, 11, 12} 这个集合; 之后调用 acpi_pci_link_get_current 函数, 运行 _CRS 代码并从 {3, 4, 5, 7, 9, 10, 11, 12} 集合中获得 irq 号。acpi_pci_link_init 函数执行完毕后, Linux 系统将显示以下信息。

```
ACPI: PCI Interrupt Link [LNKA] (IRQs 3 4 5 7 9 10 * 11 12)
ACPI: PCI Interrupt Link [LNKB] (IRQs 3 4 5 7 9 * 10 11 12)
ACPI: PCI Interrupt Link [LNKC] (IRQs 3 4 5 7 9 10 * 11 12)
ACPI: PCI Interrupt Link [LNKD] (IRQs 3 4 5 7 9 10 * 11 12)
ACPI: PCI Interrupt Link [LNKE] (IRQs 3 4 5 7 * 9 10 11 12)
ACPI: PCI Interrupt Link [LNKF] (IRQs 3 4 5 7 9 * 10 11 12)
ACPI: PCI Interrupt Link [LNKG] (IRQs 3 4 5 7 * 9 10 11 12)
ACPI: PCI Interrupt Link [LNKH] (IRQs 3 4 5 7 9 10 * 11 12)
```

其中 LNKA 使用 IRQ11, LNKB 使用 IRQ10, 并以此类推。如果一个处理器系统使能了 I/O APIC, acpi_pci_link_init 函数的执行结果并不重要, 因为 PCI 设备在执行 pci_enable_device 函数后, 该设备使用的 irq 号, 还将发生变化。

目前 Linux x86 系统在大多数情况下, 都会使能 I/O APIC, 在这种情况下, 即便不执行 acpi_pci_link_add 函数对系统也没有什么影响, 也正是基于这个考虑, 本节对 acpi_pci_link_init 函数并不做深入研究。

15.2 使用 MSI/MSIX 中断机制申请中断向量

上文讲述了 ACPI 如何为 PCI 设备或者“使用 INTx Emulation 方式”的 PCIe 设备分配中断向量。本节讲述 PCIe 设备使用 MSI/MSIX 中断机制时，Linux 系统如何分配中断向量。对于 PCI 设备，MSI/MSIX 中断机制是可选的，但是 PCIe 设备必须支持 MSI 或者 MSI-X 中断机制，或者同时支持这两种中断机制。

15.2.1 Linux 如何使能 MSI 中断机制

如果 PCI/PCIe 设备需要使用 MSI 中断机制，将调用 `pci_enable_msi` 函数，在 Linux 2.6.31 内核中，`pci_enable_msi` 函数使用 `pci_enable_msi_block(pdev, 1)` 实现。`pci_enable_msi_block` 函数在 `./drivers/pci/msi.c` 文件中，如源代码 15-8 所示。`pci_enable_msi_block` 函数具有两个入口参数，其中 `dev` 参数存放 PCIe 设备的 `pci_dev` 结构，而 `nvec` 参数为申请的 irq 号个数。

该函数返回值为 0 时，表示成功返回，此时该函数将更新 `pci_dev→irq` 参数，此时在 Linux 设备驱动程序中，可以使用的 irq 号在 `pci_dev→irq ~ pci_dev→irq+nvec-1` 之间；当函数返回值为负数时，表示出现错误；而为正数时，表示 `pci_enable_msi_block` 函数没有成功返回，返回值为该 PCIe 设备 MSI Capabilities 结构的 Multiple Message Capable 字段。

源代码 15-8 `pci_enable_msi_block` 函数

```
int pci_enable_msi_block(struct pci_dev *dev, unsigned int nvec)
{
    int status, pos, maxvec;
    u16 msgctl;

    pos = pci_find_capability(dev, PCI_CAP_ID_MSI);
    if (! pos)
        return -EINVAL;
    pci_read_config_word(dev, pos + PCI_MSI_FLAGS, &msgctl);
    maxvec = 1 << ((msgctl & PCI_MSI_FLAGS_QMASK) >> 1);
    if (nvec > maxvec)
        return maxvec;

    status = pci_msi_check_device(dev, nvec, PCI_CAP_ID_MSI);
    if (status)
        return status;

    WARN_ON(! dev->msi_enabled);

    /* Check whether driver already requested MSI-X irqs */
    if (dev->msix_enabled) {
```



```

        dev_info(&dev->dev, "can't enable MSI "
                "(MSI-X already enabled)\n");
        return -EINVAL;
    }

    status = msi_capability_init( dev, nvec );
    return status;
}

```

这段代码首先检查 PCI 设备是否支持 MSI 中断机制，如果不支持将直接退出该函数。否则检查 nvec 参数和 Multiple Message Capable 字段的大小，如果 nvec 的值较大时，该函数直接使用 Multiple Message Capable 字段返回。

如果 pci_enable_msi_block 函数通过了这些检查，将调用 pci_msi_check_device 函数，检查 Linux 系统是否能够使能 PCI 设备的 MSI 中断机制。这个检查包含两方面内容，一方面是纯软件层面的，包括检查全局变量 pci_msi_enable、pci_dev→no_msi 参数等；一方面是硬件层面的检测，包括当前 PCI 设备的上游 PCI 桥是否支持 MSI 报文的转发，PCI 设备是否具有 Capabilities 链表，是否具有 MSI Capability 结构。完成这些检查后，pci_enable_msi 将进一步调用 msi_capability_init 函数，完成与 MSI 中断相关的设置，msi_capability_init 函数的实现如源代码 15-9、15-10 所示。

源代码 15-9 msi_capability_init 函数片段 1

```

static int msi_capability_init( struct pci_dev * dev, int nvec )
{
    struct msi_desc * entry;
    int pos, ret;
    u16 control;
    unsigned mask;

    pos = pci_find_capability( dev, PCI_CAP_ID_MSI );
    msi_set_enable( dev, pos, 0 ); /* Disable MSI during set up */

    pci_read_config_word( dev, msi_control_reg( pos ), &control );
    /* MSI Entry Initialization */
    entry = alloc_msi_entry( dev );
    if ( ! entry )
        return -ENOMEM;

    entry->msi_attrib.is_msix = 0;
    entry->msi_attrib.is_64 = is_64bit_address( control );
    entry->msi_attrib.entry_nr = 0;
    entry->msi_attrib.maskbit = is_mask_bit_support( control );
}

```

```

entry->msi_attrib.default_irq = dev->irq; /* Save IOAPIC IRQ */
entry->msi_attrib.pos = pos;

entry->mask_pos = msi_mask_reg(pos, entry->msi_attrib.is_64);
/* All MSIs are unmasked by default, Mask them all */
if (entry->msi_attrib.maskbit)
    pci_read_config_dword(dev, entry->mask_pos, &entry->masked);
mask = msi_capable_mask(control);
msi_mask_irq(entry, mask, mask);

list_add_tail(&entry->list, &dev->msi_list);

```

msi_capability_init 函数具有两个入口参数，在 Linux 2.6.30 内核中，该函数具有一个入口参数，仅能获得一个 irq 号。msi_capability_init 函数参考了 msix_capability_init 函数的实现机制。这段代码置 PCI 设备的 MSI Capability 结构的 Enable 位为 0，msi_capability_init 函数需要对 MSI Capability 结构进行读写操作，因此需要暂时禁止当前设备使用 MSI 中断机制。

这段程序随后读取 MSI Capability 结构的 Message Control 字段，并暂时保存在 control 变量中，在 control 变量中存放 PCIe 设备使用的 MSI Capability 结构的格式，如图 10-1 所示，MSI Capability 结构可以使用 4 种格式。

最后这段程序调用 alloc_msi_entry 函数分配一个 msi_desc 结构的 entry 参数，并将其初始化后，加入到 pci_dev->msi_list 链表中。在 entry 参数中存放该 PCI 设备使用的 MSI 中断机制的详细信息。

源代码 15-10 msi_capability_init 函数片段 2

```

/* Configure MSI capability structure */
ret = arch_setup_msi_irqs(dev, nvec, PCI_CAP_ID_MSI);
if (ret) {
    msi_mask_irq(entry, mask, ~mask);
    msi_free_irqs(dev);
    return ret;
}

/* Set MSI enabled bits */
pci_intx_for_msi(dev, 0);
msi_set_enable(dev, pos, 1);
dev->msi_enabled = 1;

dev->irq = entry->irq;
return 0;
}

```

这段代码继续调用 arch_setup_msi_irqs 函数设置 MSI Capability 结构的其他字段，并设置

entry 结构的 irq 参数，arch_setup_msi_irqs 函数的实现与体系结构相关，下文将分别介绍 x86 和 PowerPC 处理器的实现方式。

然后这段代码调用 pci_intx_for_msi 函数，关闭 PCI 设备配置空间 Command 寄存器的 Interrupt Disable 位，因为该 PCI 设备将使用 MSI 中断机制，而不是传统的 INTx 中断机制；并调用 msi_set_enable 函数使能 MSI Capability 结构的 Enable 位；最后对 pci_dev→msi_enabled 位置 1，并将 pci_dev→irq 参数赋值。

1. Linux x86

Linux x86 使用的 arch_setup_msi_irqs 函数在 ./arch/x86/kernel/apic/io_apic.c 文件中，其实现如源代码 15-11 所示，本节并不关心 intr_remapping_enabled 参数为 1 的情况，该参数与 IOMMU 机制的 IRQ Remapping 相关。

源代码 15-11 Linux x86 使用的 arch_setup_msi_irqs 函数

```
int arch_setup_msi_irqs(struct pci_dev *dev, int nvec, int type)
{
    /* x86 doesn't support multiple MSI yet */
    if (type == PCI_CAP_ID_MSI && nvec > 1)
        return 1;

    node = dev_to_node(&dev->dev);
    irq_want = nr_irqs_gsi;
    sub_handle = 0;
    list_for_each_entry(msidesc, &dev->msi_list, list) {
        irq = create_irq_nr(irq_want, node);
        if (irq == 0)
            return -1;
        irq_want = irq + 1;
        if (!intr_remapping_enabled)
            goto no_irq;
        ...
    no_irq:
        ret = setup_msi_irq(dev, msidesc, irq);
        if (ret < 0)
            goto error;
        sub_handle++;
    }
    return 0;

error:
    destroy_irq(irq);
    return ret;
}
```

这段代码首先判断 type 是否为 PCI_CAP_ID_MSI，而且 nvec 参数是否大于 1，如果满足这两个条件，该函数将直接返回 1。通过这段代码可以发现，虽然在 Linux 2.6.31 内核中定义了一个新的 pci_enable_msi_block 函数，但是 PCIe 设备依然只能使用一个中断向量号。

这段程序随后调用 create_irq_nr 函数，分配 PCI 设备使用的 irq 号，并将其保存到 irq 变量中，之后调用 setup_msi_irq 函数初始化当前 PCI 设备的 MSI Capability 结构。setup_msi_irq 函数是一个重要函数，其实现如源代码 15-12 所示。

源代码 15-12 setup_msi_irq 函数

```
static int setup_msi_irq(struct pci_dev *dev, struct msi_desc *msidesc, int irq)
{
    int ret;
    struct msi_msg msg;

    ret = msi_compose_msg(dev, irq, &msg);
    if (ret < 0)
        return ret;

    set_irq_msi(irq, msidesc);
    write_msi_msg(irq, &msg);

    if (irq_remapped(irq)) {
        struct irq_desc *desc = irq_to_desc(irq);
        ...
        desc->status |= IRQ_MOVE_PCNTXT;
        set_irq_chip_and_handler_name(irq, &msi_irq_chip,
                                     handle_edge_irq, "edge");
    } else
        set_irq_chip_and_handler_name(irq, &msi_chip,
                                     handle_edge_irq, "edge");

    dev_printk(KERN_DEBUG, &dev->dev, "irq %d for MSI/MSI-X\n", irq);

    return 0;
}
```

这段代码首先调用 msi_compose_msg 函数，初始化 msg 结构的 address_hi、address_lo 和 data 参数，与 MSI Capability 结构的 Message Upper Address、Message Address 和 Message Data 字段对应。

对于 x86 处理器系统，Message Address 字段的格式见图 10-1，其中 Destination ID 字段与 CPU 的 ACPI ID 相关，Linux x86 使用 cpu_mask_to_apicid_and 函数获得该字段的值；而 Message Data 字段的格式如图 10-1 所示，其 Vector 字段由 assign_irq_vector 函数设置，

Trigger Mode 字段为 0x00 表示使用边沿触发方式，而 Delivery Mode 字段为“Fixed Mode”或者“Lowest Priority”。值得注意的是，在 Message Data 字段中存放的是中断向量号（vector），是一个硬件的概念，而设备驱动程序中使用的 irq 号是一个软件关系，两者之间存在对应关系，但是并不等同。

set_irq_msi 函数设置 PCIe 设备使用的 irq_desc 结构；而 write_msi_msg 函数将 msg 结构中的参数写入到 PCIe 设备的对应寄存器中；而 set_irq_chip_and_handler_name 函数设置 MSI 中断使用的中断处理程序。本节对这些函数不做进一步介绍。

2. PowerPC

Linux PowerPC 使用的 arch_setup_msi_irqs 函数在 ./arch/powerpc/kernel/msi.c 文件中，对于 Freescale 的 PowerPC 处理器，该函数等效与 fsl_setup_msi_irqs。fsl_setup_msi_irqs 函数的实现，如源代码 15-13 所示。

源代码 15-13 fsl_setup_msi_irqs 函数

```
static int fsl_setup_msi_irqs(struct pci_dev *pdev, int nvec, int type)
{
    ...
    list_for_each_entry(entry, &pdev->msi_list, list) {
        hwirq = msi_bitmap_alloc_hwirqs(&msi_data->bitmap, 1);
        ...
        virq = irq_create_mapping(msi_data->irqhost, hwirq);
        ...
        set_irq_msi(virq, entry);
        fsl_compose_msi_msg(pdev, hwirq, &msg);
        write_msi_msg(virq, &msg);
    }
    return 0;
out_free:
    return rc;
}
```

该函数的实现机制与 x86 处理器类似，值得提醒读者注意的是在 fsl_compose_msi_msg 函数中 msg->address_hi 等于 fsl_msi->msi_addr_lo，其值为 MSIIR 寄存器在 PCI 总线域的物理地址。在该函数中使用的 address_hi、address_lo 和 data 参数的详细描述见第 10.2 节。本节对此不一一叙述。目前在 PowerPC 处理器系统中，PCIe 设备也只能使用一个中断向量号。

15.2.2 Linux 如何使能 MSI-X 中断机制

在 Linux 系统中，如果 PCI/PCIe 设备需要使用 MSI-X 中断机制，需要调用 pci_enable_msix 函数，pci_enable_msix 函数调用的大多数函数与 pci_enable_msi 类似，本节并不会重复解释这些函数，该函数的实现如源代码 15-14 所示。

源代码 15-14 pci_enable_msix 函数

```

int pci_enable_msix(struct pci_dev * dev, struct msix_entry * entries, int nvec)
{
    int status, nr_entries;
    int i, j;

    if (! entries)
        return -EINVAL;

    status=pci_msi_check_device(dev, nvec, PCI_CAP_ID_MSIX);
    if (status)
        return status;

    nr_entries=pci_msix_table_size(dev);
    if (nvec > nr_entries)
        return nr_entries;

    /* Check for any invalid entries */
    for (i=0; i < nvec; i++) {
        if (entries[i].entry >= nr_entries)
            return -EINVAL; /* invalid entry */
        for (j=i + 1; j < nvec; j++) {
            if (entries[i].entry == entries[j].entry)
                return -EINVAL; /* duplicate entry */
        }
    }
    WARN_ON(!! dev->msix_enabled);

    /* Check whether driver already requested for MSI irq */
    if (dev->msi_enabled) {
        dev_info(&dev->dev, "can't enable MSI-X "
                "(MSI IRQ already assigned)\n");
        return -EINVAL;
    }
    status=msix_capability_init(dev, entries, nvec);
    return status;
}

```

与 pci_enable_msi_block 函数不同, pci_enable_msix 函数的入口参数包括一个 msix_entry 结构的 entries 链表 (在使用这个 entries 链表之前需要将 msix_entry.entry 参数赋值), 而 nvec 参数保存 entries 链表的长度。该函数首先对入口参数进行检查, 然后调用 msix_capability_init 函数为 PCIe 设备分配多个中断向量号。msix_capability_init 函数的实现与 msi_

capability_init 函数的实现方法类似，本章对此不做进一步描述。

该函数成功返回后，PCIe 设备将得到多个中断向量，并将结果放入 pci_dev→msi_list 和 entries 链表中，之后 PCIe 设备的 Linux 驱动程序可以使用多个 request_irq 函数注册相应的中断服务例程。

下文将以 Intel 的 e1000e 网卡驱动程序说明如何使用 MSI-X 中断机制挂接中断服务例程。在 Linux 中，与 e1000e 网卡相关的驱动程序在 ./drivers/net/e1000e/netdev.c 文件中。其中 MSI-X 中断机制的初始化在 e1000_probe→e1000_sw_init→e1000e_set_interrupt_capability 函数中，该函数的实现如源代码 15-15 所示。

源代码 15-15 e1000e_set_interrupt_capability 函数

```
void e1000e_set_interrupt_capability(struct e1000_adapter *adapter)
{
    ...
    switch (adapter->int_mode) {
    case E1000E_INT_MODE_MSIX:
        if (adapter->flags & FLAG_HAS_MSIX) {
            numvecs = 3; /* RxQ0, TxQ0 and other */
            adapter->msix_entries = kcalloc(numvecs,
                                           sizeof(struct msix_entry),
                                           GFP_KERNEL);
            if (adapter->msix_entries) {
                for (i = 0; i < numvecs; i++)
                    adapter->msix_entries[i].entry = i;
                err = pci_enable_msix(adapter->pdev,
                                     adapter->msix_entries,
                                     numvecs);
                if (err == 0)
                    return;
            }
        }
        ...
    }
}
```

当 e1000e_set_interrupt_capability 函数返回后，MSI-X 中断机制使用的中断向量将被保存在 adapter->msix_entries 数组中，之后 e1000_open→e1000_request_irq→e1000_request_msix 函数将多次调用 request_irq 函数将 e1000e 使用的中断服务例程挂接到系统中断服务程序中，e1000_request_msix 函数的实现如源代码 15-16 所示。

源代码 15-16 e1000_request_msix 函数

```
static int e1000_request_msix(struct e1000_adapter *adapter)
{
    ...
}
```

```
...
    err=request_irq( adapter->msix_entries[ vector]. vector,
                    &e1000_intr_msix_rx, 0, adapter->rx_ring->name,
                    netdev);

...
    err=request_irq( adapter->msix_entries[ vector]. vector,
                    &e1000_intr_msix_tx, 0, adapter->tx_ring->name,
                    netdev);

...
    err=request_irq( adapter->msix_entries[ vector]. vector,
                    &e1000_msix_other, 0, netdev->name, netdev);

...
}
```

e1000_request_msix 函数将“接收完成中断请求 e1000_intr_msix_rx”“发送完成中断请求 e1000_intr_msix_tx”和“其他中断请求 e1000_msix_other”分别注册。当有中断事件发生时，驱动程序不需要读取中断状态寄存器之后再进行处理，从而有效降低了系统延时。

15.3 小结

本节主要介绍了 PCI 设备的中断请求在 Linux 系统中的处理过程。Linux 系统的更新速度较快，并不断加入新的功能。本章的内容基于 Linux 系统，目前 Linux 系统支持多种架构的处理器系统，而且得到了极大的普及。对于有志于学习体系结构的工程师而言，深入了解几种操作系统是必须的。而在这些操作系统中，Linux 无疑最为开放，读者也最容易了解其实现细节。但是值得注意的是，在体系结构的学习过程中，不要拘泥于 Linux 系统本身，Linux 系统仅包含了体系结构的部分内容，也只是一种实现方法。

本书到此告一段落，而 PCIe 总线仍然在发展，PCIe V3.0 规范即将发布，其中增加了许多新的功能，而这些新的功能在许多处理器系统中并没有意义。这些新的功能在本书中多有提及，但并不是本书的重点。本书的重点是以 PCIe 总线为例说明处理器的体系结构，是对 PCIe 体系结构进行导读，更准确地说，是以 PCIe 总线为例说明处理器体系结构中局部总线的设计原理与使用方法。